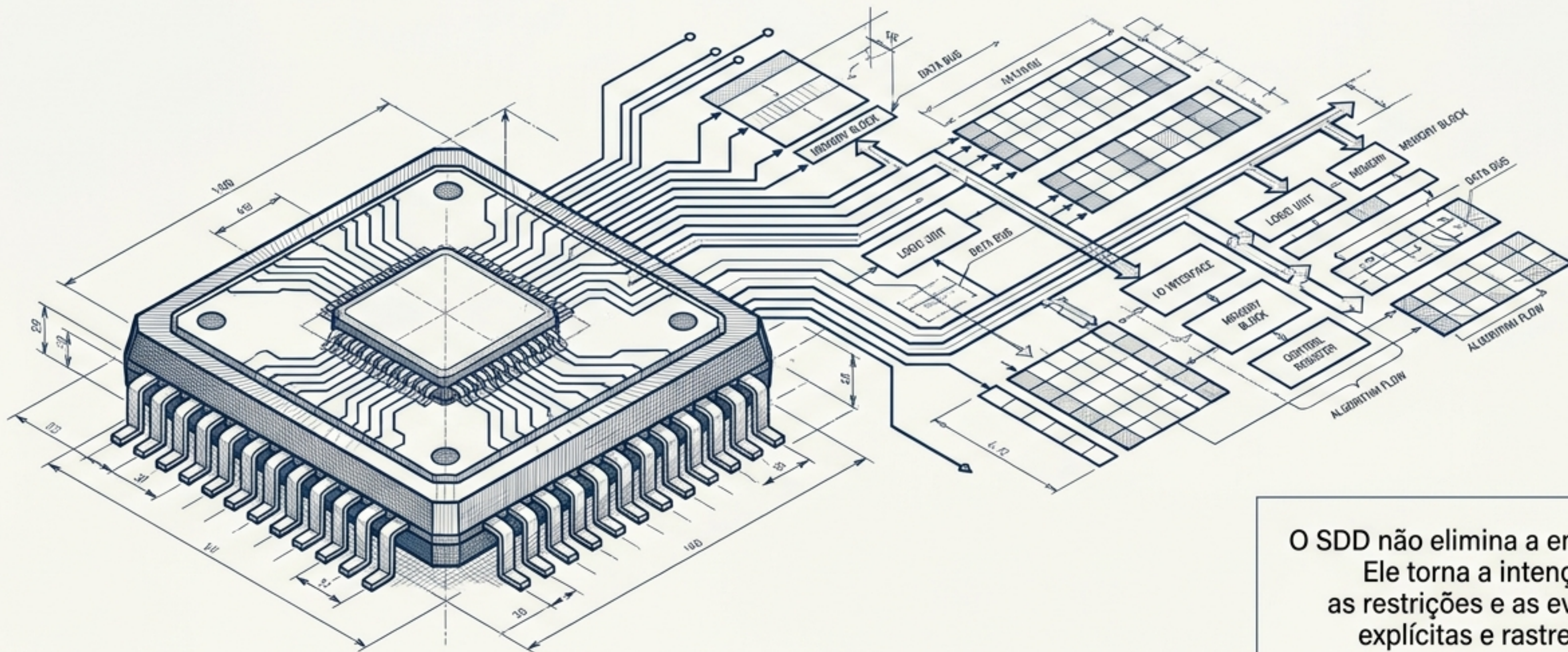


SDD para Software Embarcado

Da especificação ao firmware em microcontroladores



O SDD não elimina a engenharia.
Ele torna a intenção,
as restrições e as evidências
explícitas e rastreáveis.

SE A IA ESCREVE O CÓDIGO, O PROGRAMADOR ACABOU?

NÃO.

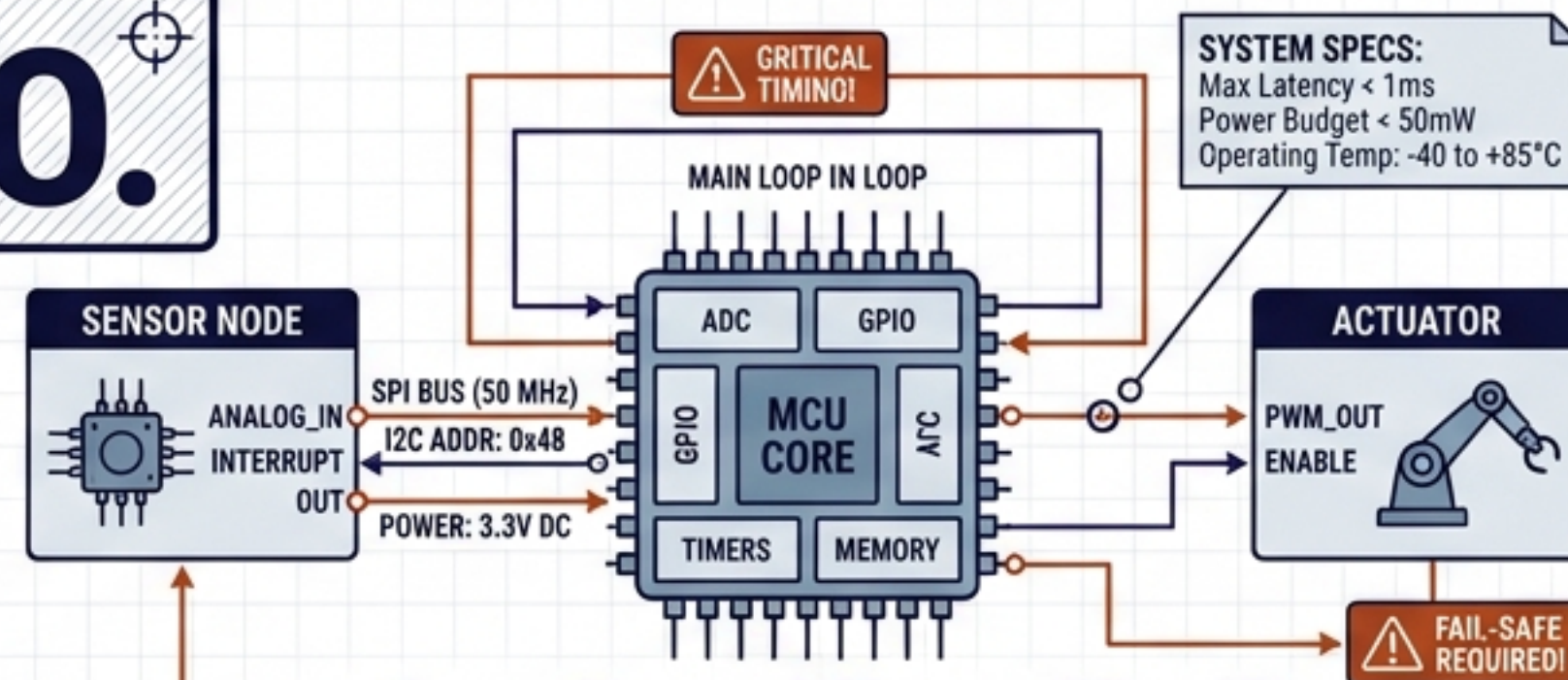
```
#include <stdio.h>
#include <stdbool.h>
#include "system_config.h"
```

```
// AI generated function for data processing
int process_sensor_data(int raw_value) {
    if (raw_value < MIN_THRESHOLD) {
        return 0; // Below threshold
    } else if (raw_value > MAX_THRESHOLD) {
        return MAX_OUTPUT; // Saturation
    } else {
        // Linear scaling
        return (raw_value - MIN_THRESHOLD) * SCALE_FACTOR;
    }
}
```

```
void main() {
    system_init();
    while (true) {
        int input = read_sensor();
        int output = process_sensor_data(input);
        write_actuator(output);
        delay_ms(SAMPLE_PERIOD);
    }
}
```

A IA ESCREVE A SINTAXE.

O ENGENHEIRO DEFINE O COMPORTAMENTO.



CONSTRAINT DASHBOARD

- | | |
|--|---|
| ☒ QUAIS SÃO OS LIMITES FÍSICOS?
Sensor range, actuator power, thermal constraints | ☒ QUAL É O ESTADO SEGURO NO RESET?
Initialization sequence, output safe-states |
| ☒ QUAL É O TIMING EXATO?
Real-time deadlines, interrupt latency, jitter | ☒ QUANTO DE RAM/FLASH É CONSUMIDO?
Memory footprint, stack usage, optimizations |
| ☒ O QUE ACONTECE QUANDO O SENSOR FALHA?
Error handling, default values, system degradation | |
| ☒ QUAL É O ESTADO SEGURO NO RESET?
Initialization sequence, output safe-states | |

O SDD MUDA O FOCO DE PEDIR CÓDIGO PARA DEFINIR E VERIFICAR COMPORTAMENTO

LLMs no Mundo Embarcado

O que a LLM domina

- ✓ Interpretar especificações técnicas.
- ✓ Propor arquiteturas de software.
- ✓ Gerar código (C/C++/Rust/MicroPython).
- ✓ Escrever testes e analisar logs.
- ✓ Encontrar inconsistências lógicas.

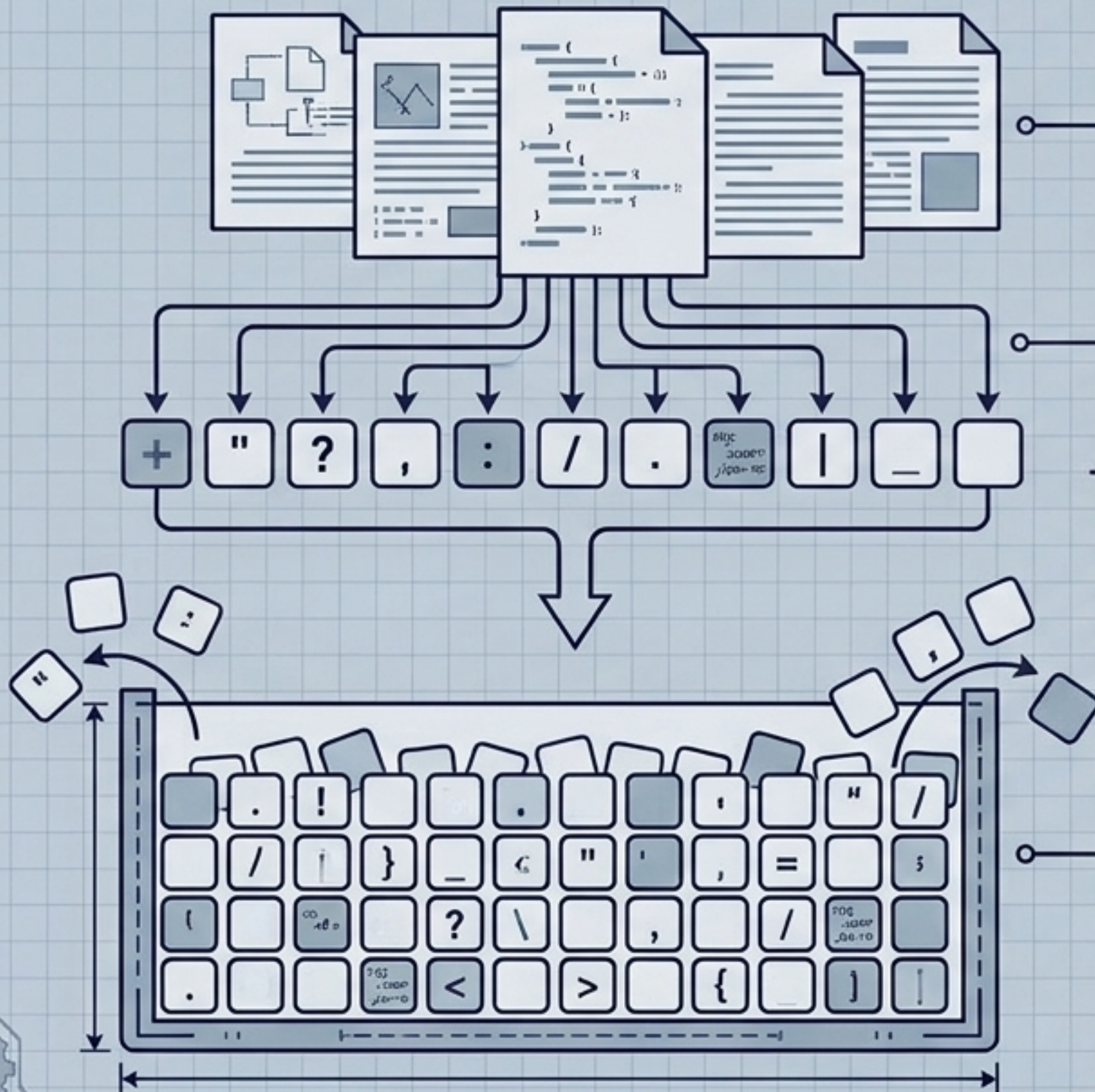
O Ponto Cego Crítico



Ela não conhece automaticamente o seu hardware real.

A IA opera prevendo a próxima sequência lógica de tokens, mas não sabe se o pino GPIO4 do seu chip está fisicamente queimado ou se o clock real da placa diverge do datasheet.

A Anatomia da IA: Tokens e Janela de Contexto



• A IA não lê palavras.

• **0 Token:** Partes de palavras, pontuação, trechos de código e espaços frequentes.

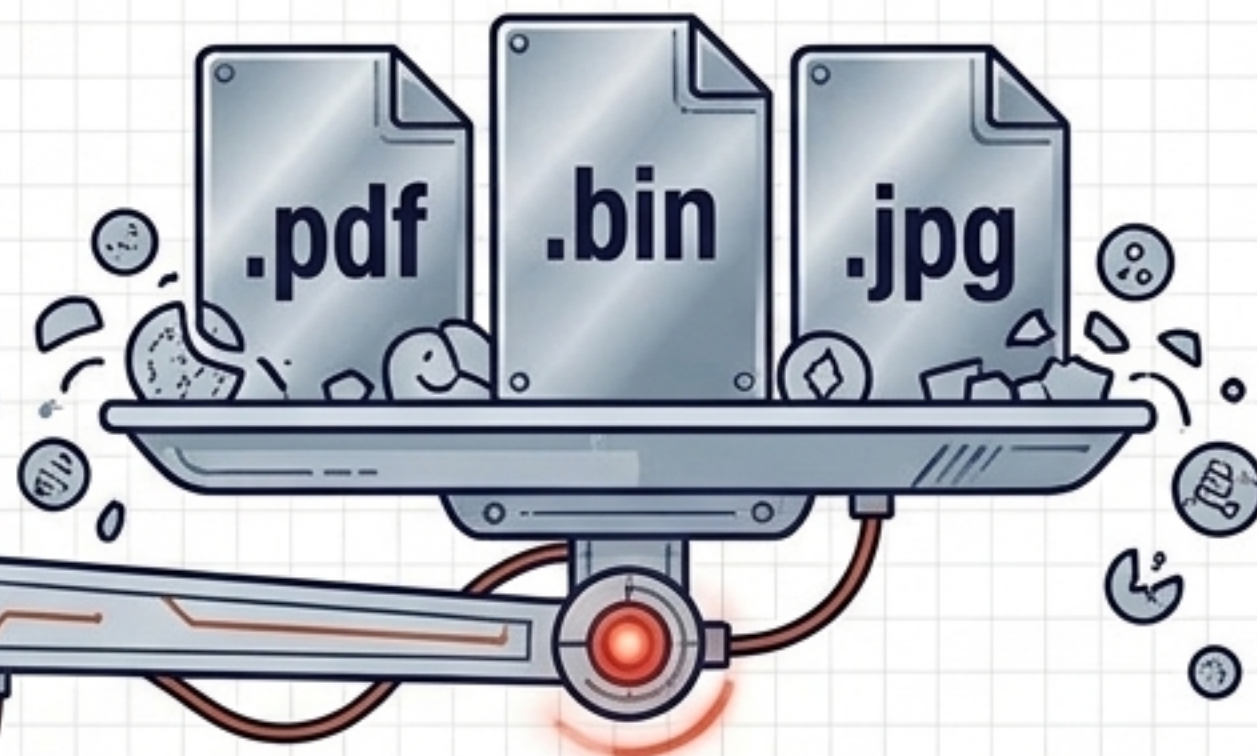
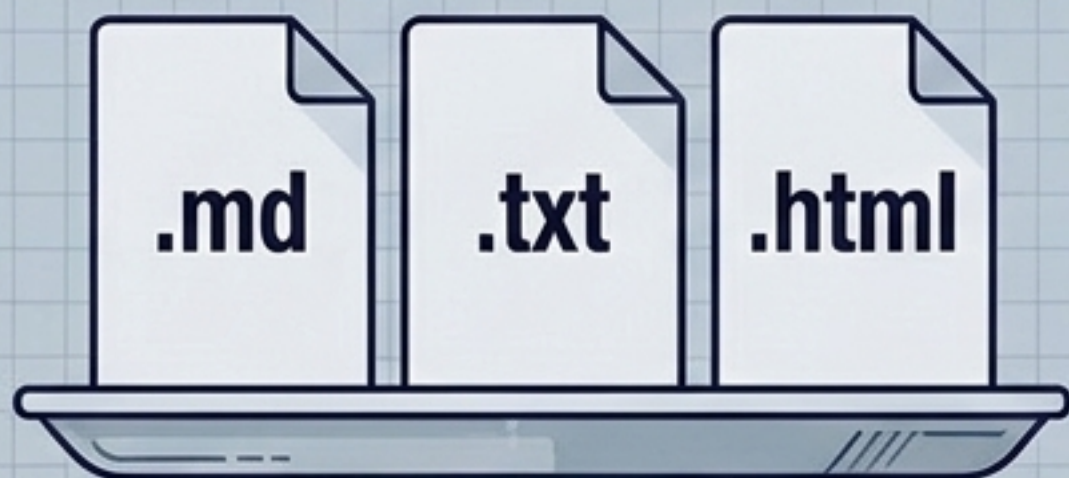
• **A Janela de Contexto:** O limite máximo de tokens que o modelo consegue lembrar em uma única interação.

Mais contexto não significa melhor resposta.

Contexto irrelevante aumenta o custo, reduz a eficiência e faz o modelo esquecer ou alucinar detalhes cruciais da sua especificação.

A Regra de Ouro do Contexto

Forneça apenas o contexto necessário, não o repositório inteiro.



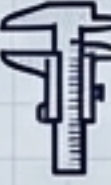
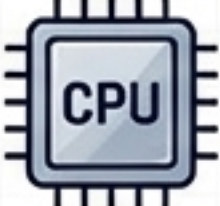
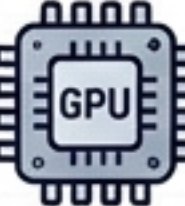
Recomendado

Prefira formatos baseados em texto puro, como Markdown ou HTML. A densidade de informação por token é maximizada.

Evitar

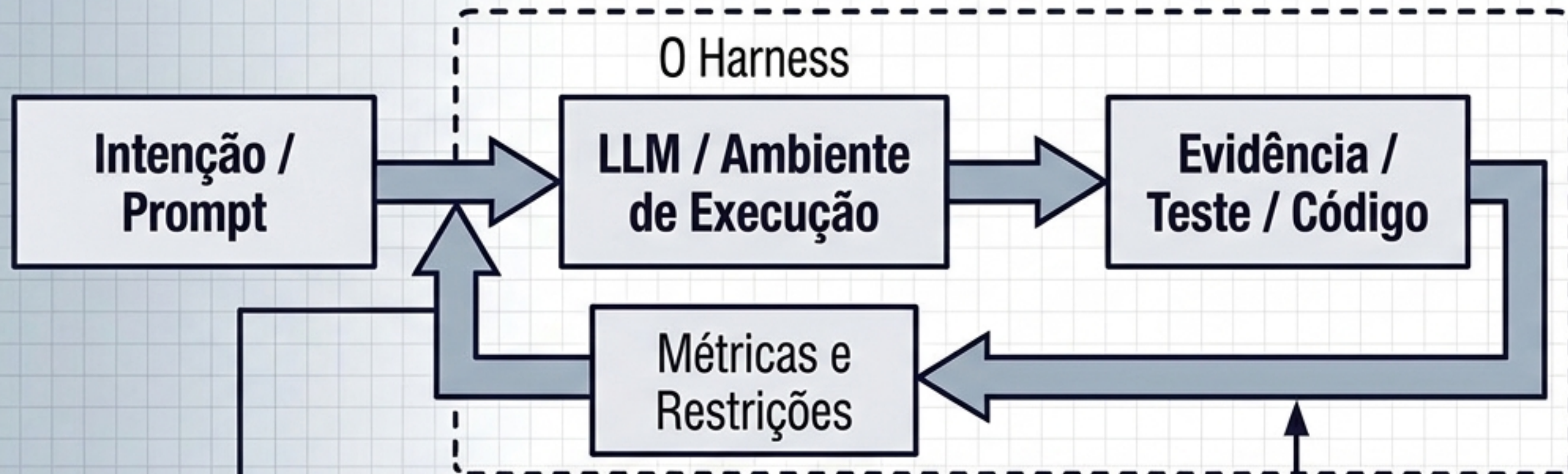
Evite enviar binários, imagens e PDFs diretamente. Eles consomem quantidades massivas de tokens processando metadados ocultos, gerando custos desnecessários sem melhorar o código.

Onde Executar? LLM Local vs. Provedor Remoto

 Dimensão		LLM Local (Máquina do Desenvolvedor)	 Provedor Remoto (Nuvem / API)
Privacidade e Dados		 Dados nunca saem da sua máquina (Ideal para IP sensível).	 Requer envio de código para servidores externos (Risco de política).
Infraestrutura		 Desempenho depende 100% da sua GPU/RAM. Pode ser lento.	 Infraestrutura pronta, acesso a modelos gigantesco instantaneamente.
Custo		 Custo fixo (energia/hardware). Previsível.	 Custo por uso (tokens processados).
Conectividade		 Funciona totalmente offline.	 Dependência estrita de internet e estabilidade do serviço.

O Elo Perdido: O Harness de Execução

Harness é a estrutura de suporte para executar, testar, medir e avaliar o sistema de forma controlada. O modelo de IA não é o harness; ele é apenas o motor.



No firmware, não perguntamos apenas
O código compilou?. Perguntamos:

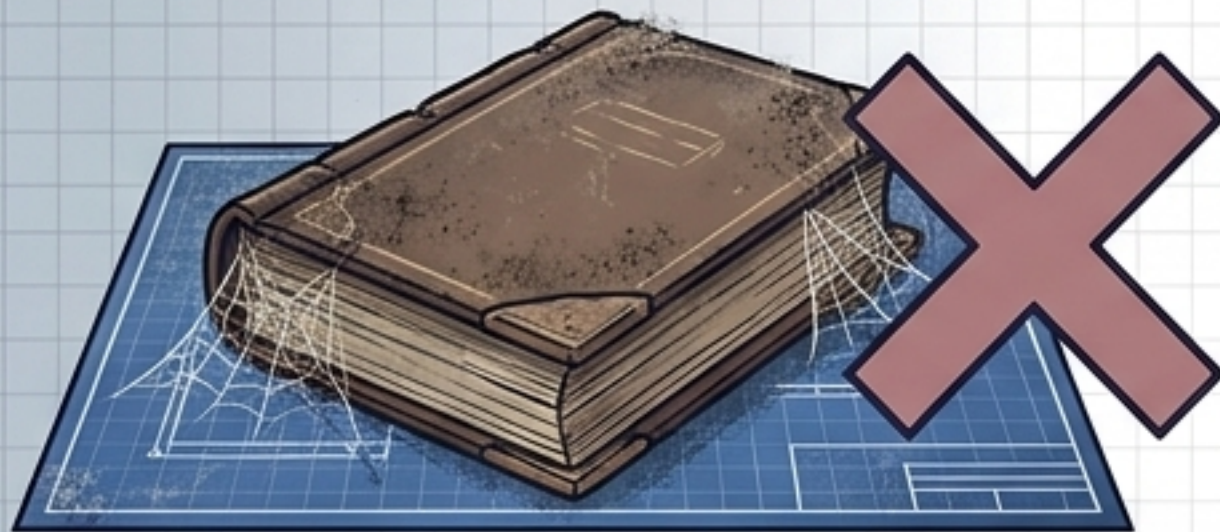
- Qual a latência exata?
- Houve jitter no timer?
- O consumo de Flash/RAM estourou?
- O Watchdog foi alimentado a tempo?

O Paradigma SDD (Specification-Driven Development)

A especificação é a única fonte da verdade e da intenção.

$$\left[\begin{array}{c} \text{Especificação} \\ \text{(A Intenção)} \end{array} \right] = \left[\begin{array}{c} \text{Código} \\ \text{(A Implementação)} \end{array} \right] + \left[\begin{array}{c} \text{Testes} \\ \text{(A Evidência)} \end{array} \right]$$

O Que NÃO É



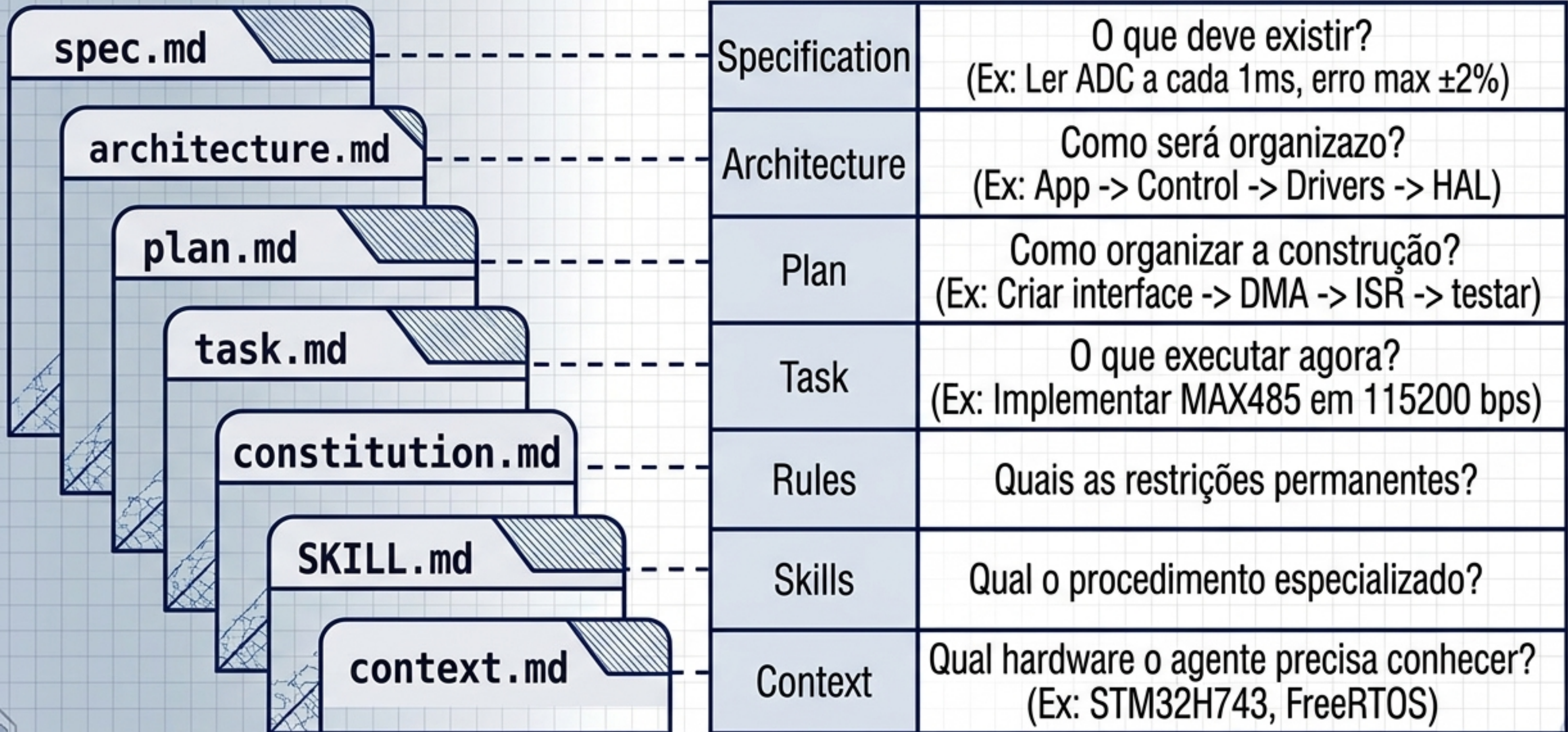
SDD não significa escrever dezenas de páginas de documentação morta.

O Que É

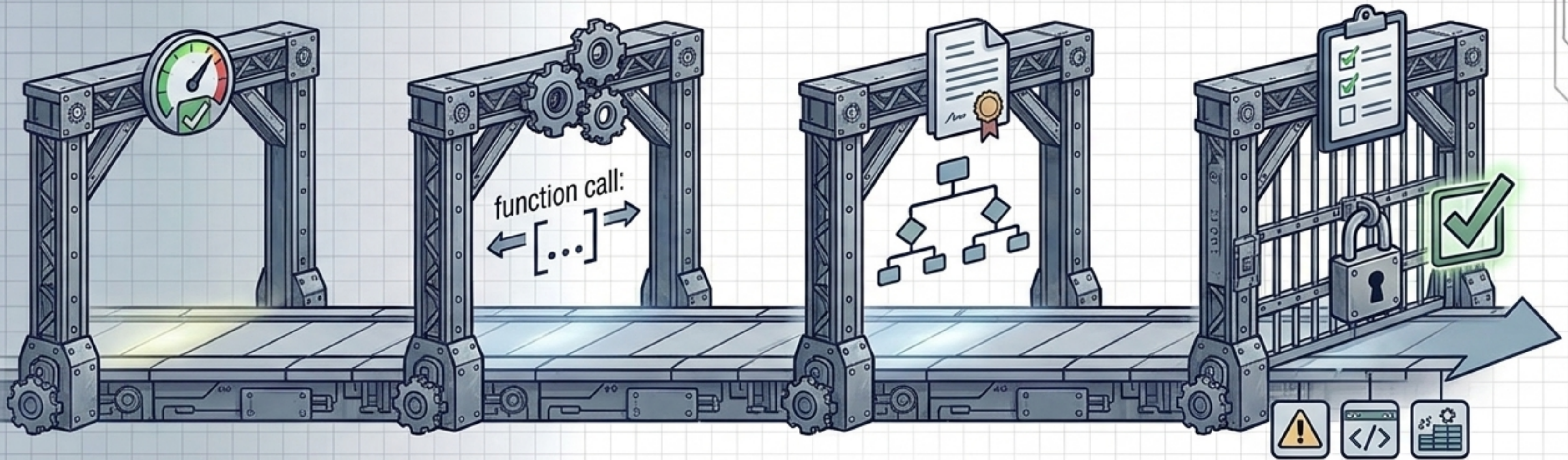


Significa definir o comportamento observável do sistema e manter essa definição estritamente conectada à implementação real e aos testes.

O Mapa de Artefatos do SDD



Fechando o Contrato: Evidências e Quality Gates



1. Acceptance Criteria

Define quando o requisito está pronto.

Ex: Manter taxa de amostragem entre 995Hz e 1005Hz por 60s.

2. Interfaces & Contracts

Como os módulos conversam.
A IA não inventa APIs.

Ex: bool
`adc_read(adc_sample_t *sample);`

3. ADRs (Decisões)

Registro imutável do “porquê”.

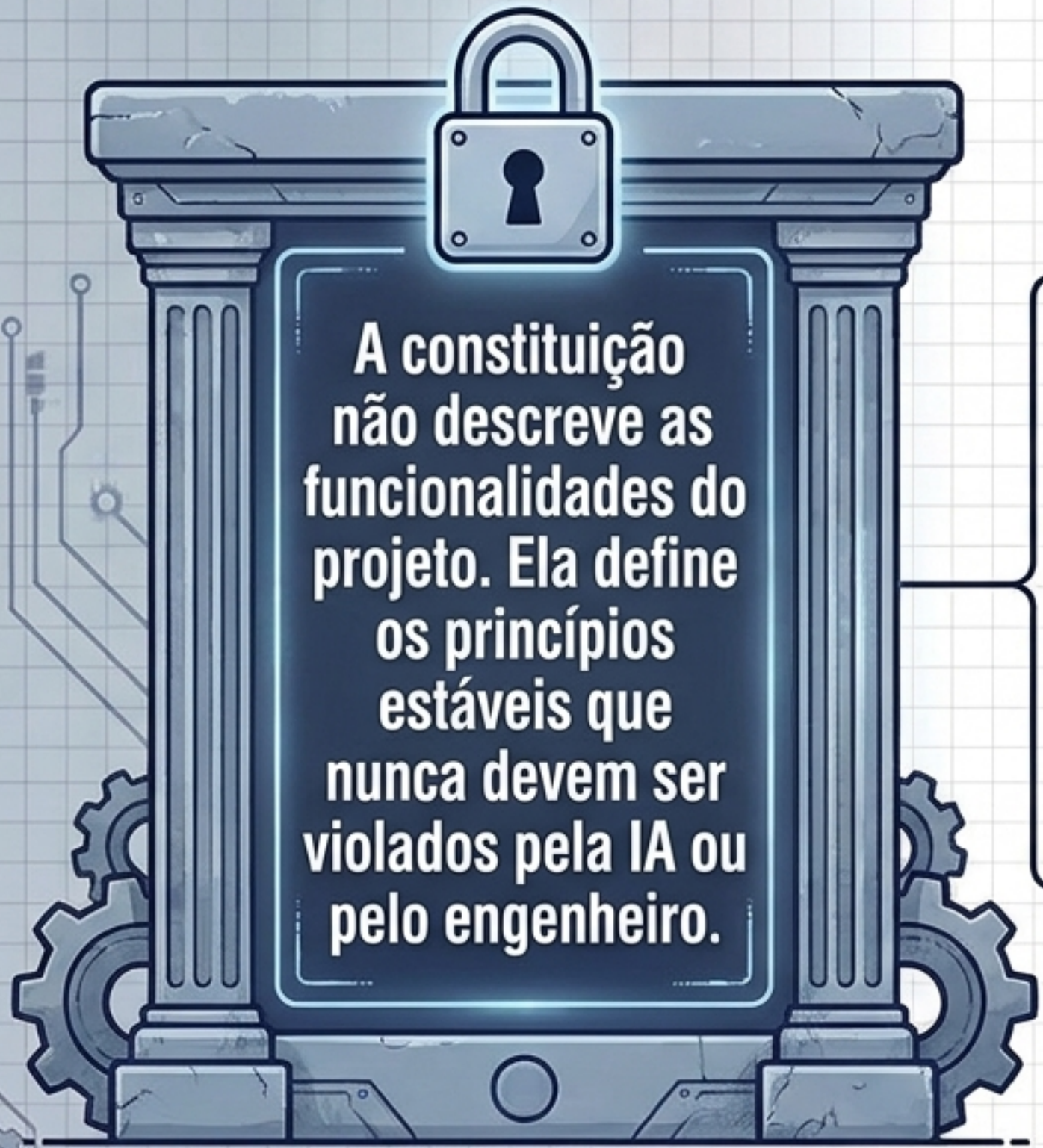
Ex: Uso de DMA em vez de polling para não bloquear a CPU.

4. Checklists

A barreira final. Impede conclusão sem evidência.

Ex: Zero warnings, análise estática limpa, teste de stack e watchdog validados.

As Regras do Jogo: O Arquivo constitution.md



A constituição não descreve as funcionalidades do projeto. Ela define os princípios estáveis que nunca devem ser violados pela IA ou pelo engenheiro.

Segurança & Estado Seguro

Fail-safes obrigatórios, comportamento da porta em falha.

Determinismo & Concorrência

Regras rígidas de prioridade do RTOS e interrupções.

Limites de Memória

Proibições rigorosas (Ex: Nunca usar alocação dinâmica / malloc).

Rastreabilidade & Diagnóstico

Padrões de logs obrigatórios para reconstrução de falhas.

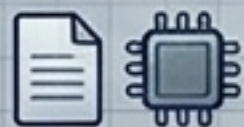
O Motor do Processo: O Arquivo SKILL.md

Se a **constitution.md** diz à IA “o que não fazer”, o SKILL.md detalha exatamente como conduzir o trabalho passo a passo.

1. Reconhecer o contexto
(Ler .md e datasheets)



1. Reconhecer
o contexto
(Ler .md e
datasheets)



2. Especificar
comportamento
detalhado



2. Especificar
comportamento
detalhado

3. Projetar arquitetura e
remover ambiguidades



3. Projetar
arquitetura e
remover
ambiguidades

4. Quebrar em
tarefas menores
(TASK.md)



4. Quebrar em tarefas
menores (TASK.md)

6. Verificar contra
os Quality Gates

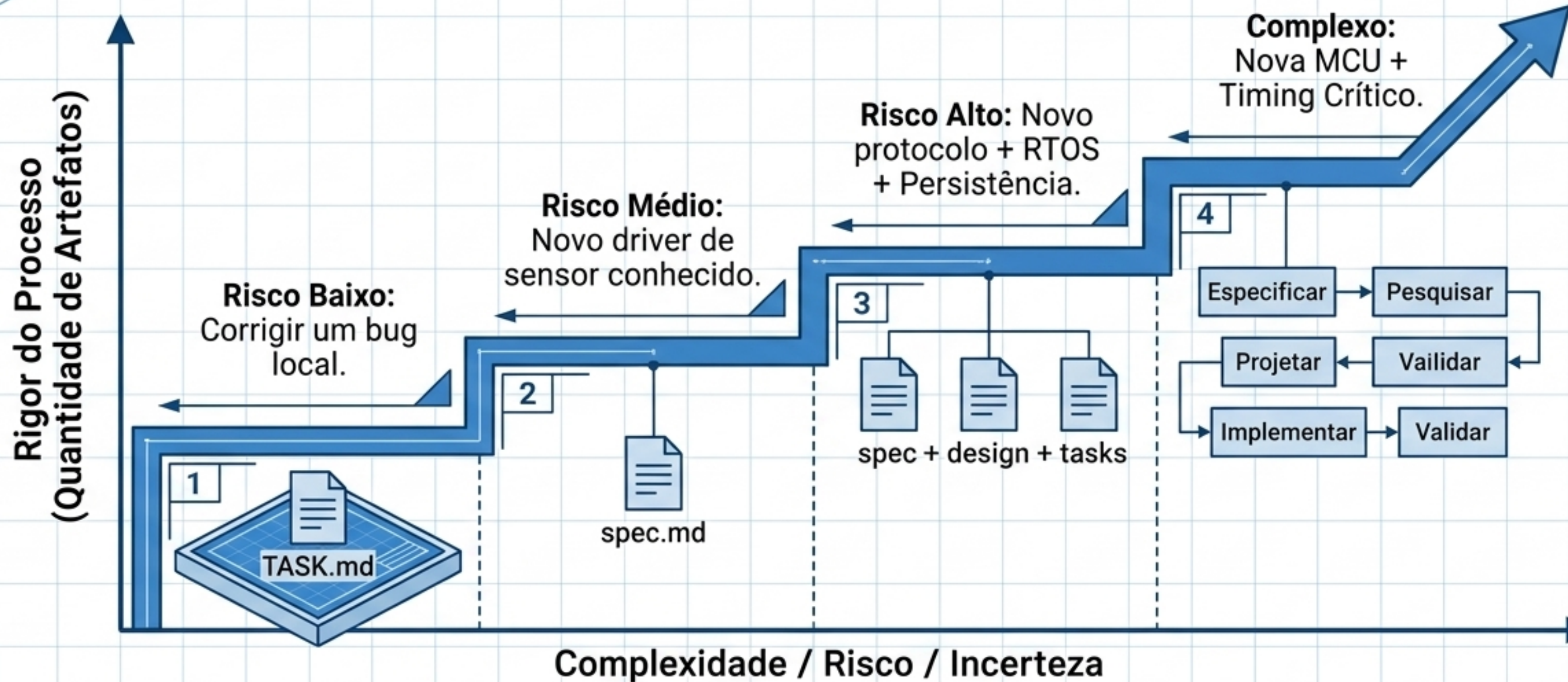


6. Verificar
contra os
Quality Gates



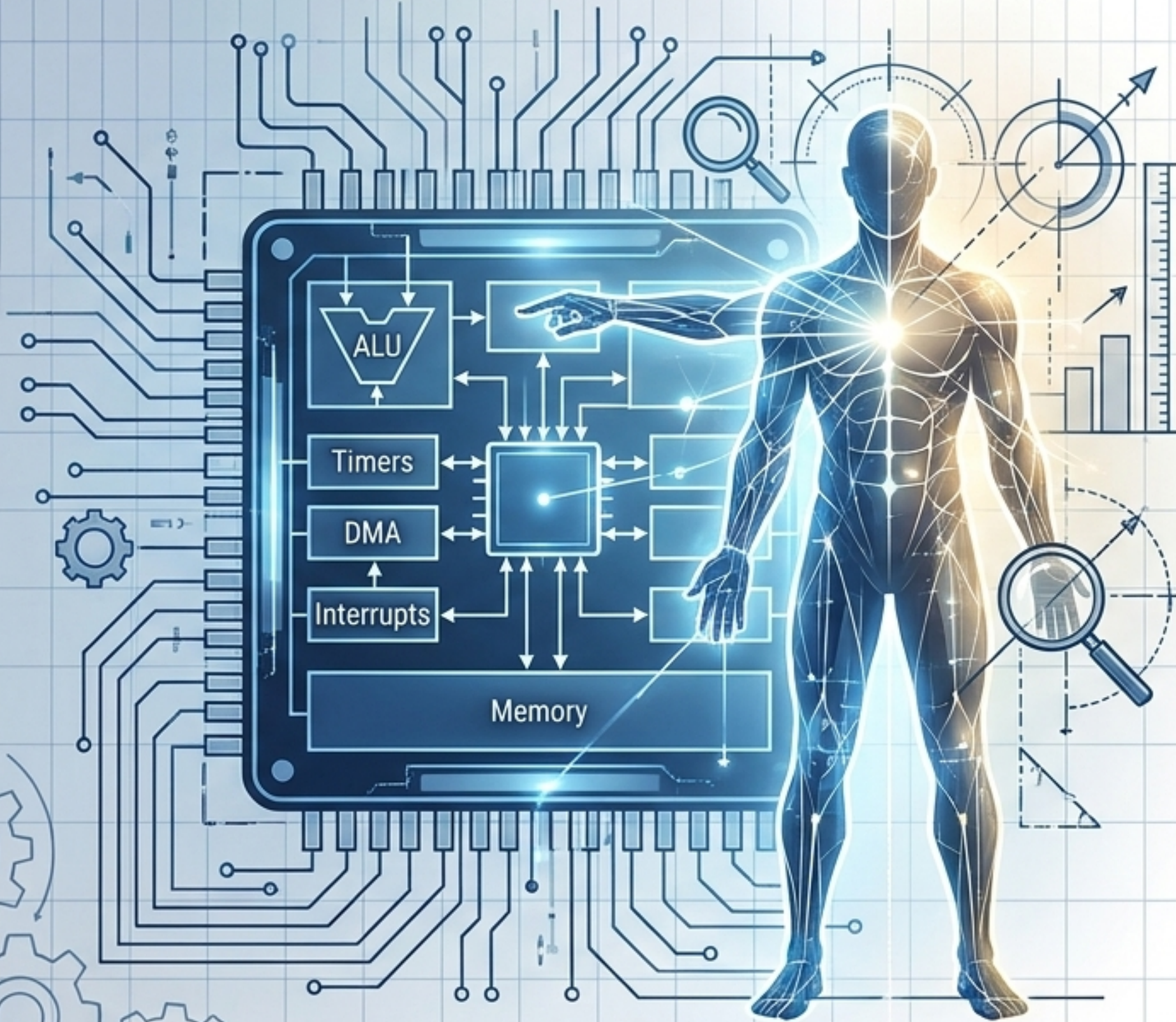
7. Ancorar e
encerrar a
sessão

SDD Adaptativo: Ajustando o Rigor



Regra de Ouro: Aumente o rigor processual sempre que aumentarem os riscos, as dependências externas ou a incerteza de hardware.

A Reflexão Final: O Diferencial do Engenheiro



A IA é capaz de codificar rápido, mas ela não conhece a física do seu hardware.

Conhecer a **arquitetura interna** da MCU (periféricos, interrupções, **registradores**) é o que separa um operador de IA de um desenvolvedor de firmware profissional.

O Valor do SDD: Com esse domínio, você cria especificações rigorosas que forçam a IA a gerar código otimizado, evitando bugs críticos de hardware.

Conclusão & Próximos Passos

Parte 1

Partes 2 & 3:
Da Teoria à
Bancada

- ☒ Limites das LLMs no firmware estabelecidos.
- ☒ Paradigma **SDD** compreendido.
- ☒ Artefatos **constitution.md** e **SKILL.md** definidos.
- Distinguir ambientes de execução: HOST, SIMULADOR, BANCADA e HIL.
- **Demonstração Prática:** Aplicação real usando VS Code + Git.
- **Programando um ESP8266** em modo AP, sensor DS18B20 (GPIO4) e dashboard web.

Preparem seus repositórios.